

•



- [Core Concepts](#)
- [Workflows](#)
- Railgun Engine

On this page

Railgun Engine

Was this page helpful? 

Railgun is Pulse's real-time [profile engine](#) that calculates aggregate data values [metrics](#) over time windows of any length with high performance, accuracy and throughput.

Due to its optimized architecture, Railgun integrates Pulse's live data processing pipeline, enriching events with historical aggregations in real time as they flow through the stages of a [workflow](#).

Profiling

In the [Feature Engineering stage of the Data Science Loop](#), data scientists can define [metrics](#) that are calculated from the existing fields of a set of events from the data stream.

Such metrics usually represent aggregated statistics. Some typical examples include: average number of transactions in the past 24 hours, or total amount spent by a customer over 30 days, or number of different payment methods used over the last 6 months.

These highly targeted metrics allow *profiling* — i.e. determining patterns in user behavior or transaction activity — in order to detect outliers and suspicious behavior. For this reason, they are called **profiles**.

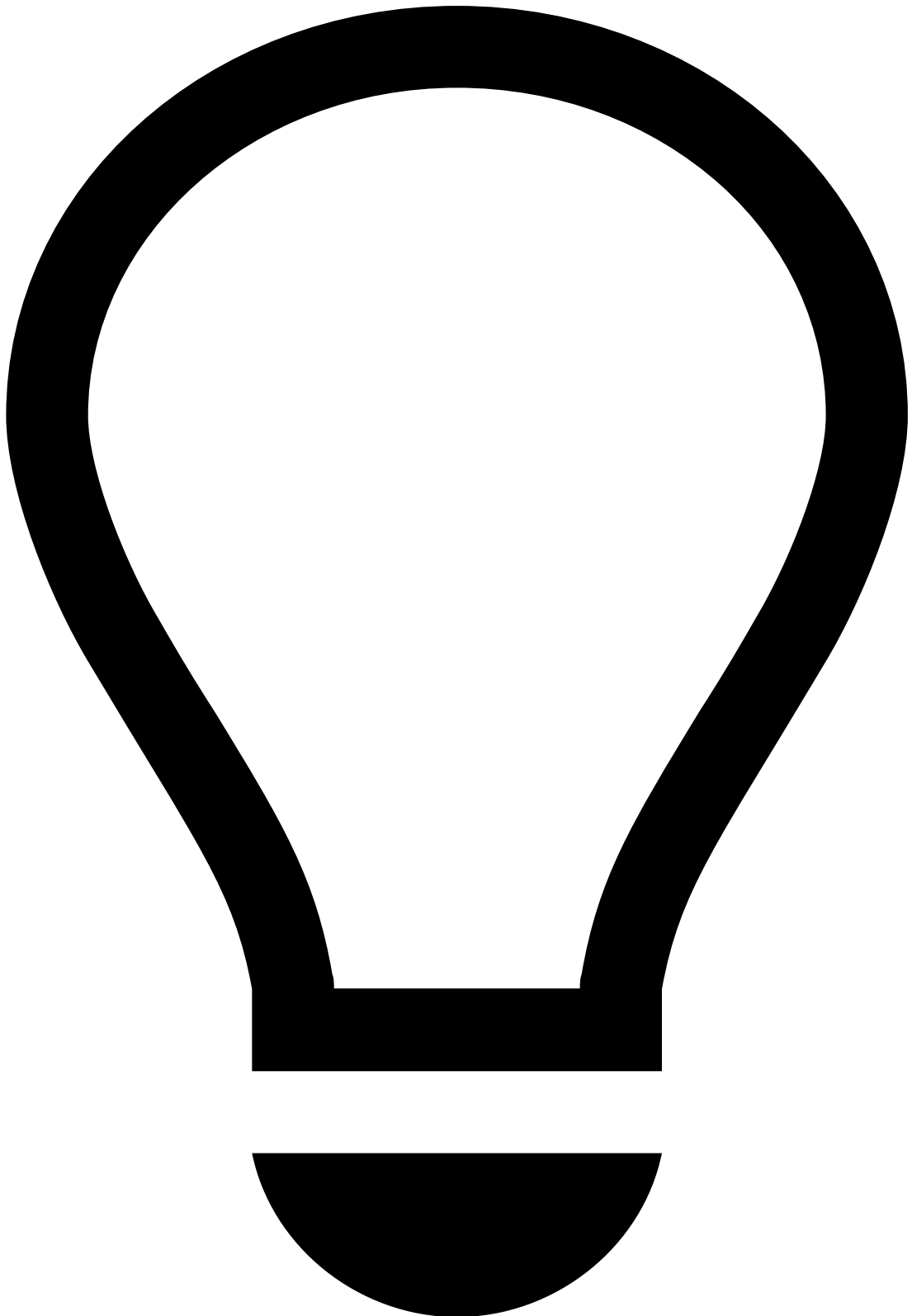
The Railgun engine has been developed specifically for calculating such profiles in an efficient manner, so they can be used when processing [events](#) in real time.

Profiles over long time windows [↗](#)

Calculating profiles for longer time windows (weeks, months or even years) is a common need in financial risk modeling, but it is typically an expensive operation since it requires collecting a potentially large amount of data if the events are frequent.

This means that real-time metrics have traditionally been limited to profiling short-term behaviors, and longer windows were handled by offline, batch jobs, i.e. [Scheduled Metrics Jobs](#) (SMJs).

Since Railgun can compute real-time metrics for long windows with low-memory consumption, it doesn't require the use of SMJs. Therefore, real-time profiles, with full responsiveness and accuracy, can be defined with arbitrarily long time windows, depending only on data retention policy agreements.



Backfills

One of the mechanisms used by Railgun to enable such efficient handling of long time windows is the strategy of making metrics immediately available with the most recent data, and then backfilling the historical data in the background. Learn more about this process in the [Backfills](#) page.

Non-windowed metrics🔖

Besides long time windows, Railgun also supports metrics that are not bound by a time window at all, being limited only by the available historical data at the time of the metric's creation or update (which is determined by the data retention policy).

Other than when creating the metric, or when updating its definition (which requires a backfill), the data retention policy does not affect the metric's temporal scope, since values calculated from data even beyond the data retention policy's window are retained in Railgun's persistent storage.

For more information on this type of metrics, refer to the [Non-windowed Metrics](#) section.

Real-time profiles

The ability to update profiles in real time, rather than as a periodic process, means that Pulse can detect changes in behavior immediately as they happen, rather than later at a scheduled profile calculation. This allows adopting a proactive risk strategy, preventing unwanted activity to take place instead of remediating after the fact, and avoiding costly reversals later. The added agility is also invaluable in thwarting rapid, automated attacks before they reach problematic scales.

With Railgun, all profiles are calculated in real time, allowing fast and convenient risk strategy updates.

Performance and scalability

Railgun's performance is achieved by processing data as a stream, handling only small batches of data at a time, while updating the aggregated values in real time on persistent storage for use in the later stages of a workflow.

This approach allows maintaining very high throughput, and seamless scalability to larger time windows with moderate memory consumption, while keeping latency consistently low.